

SECURITY AUDIT

GreenChain Protocol

Relatório de Auditoria de Segurança

Análise completa de vulnerabilidades, vetores de ataque e conformidade com boas práticas de desenvolvimento Solidity para os contratos GreenToken, CarbonCertNFT, GreenStaking e GreenDAO.

Data da Auditoria	27 de April de 2026
Versao dos Contratos	Solidity ^0.8.20
Ferramentas	Slither v0.10 · Mythril v0.24 · Hardhat v2.19 · Revisao Manual
Escopo	GreenToken · CarbonCertNFT · GreenStaking · GreenDAO
Rede de Deploy	Sepolia Testnet (Chain ID: 11155111)
Bibliotecas	OpenZeppelin v5.0 · Chainlink AggregatorV3
Classificacao	Confidencial — Residencia em TIC 29

Este relatorio foi produzido para fins academicos no contexto da Residencia em TIC 29. Nao constitui aconselhamento juridico ou financeiro. Nenhuma auditoria garante ausencia total de vulnerabilidades.

Sumario Executivo

A auditoria do protocolo GreenChain cobriu quatro contratos Solidity deployados na Sepolia Testnet. O protocolo implementa tokenizacao de creditos de carbono via NFT ERC-721, token de governanca ERC-20, staking com APR dinamico baseado em oracle Chainlink e uma DAO simplificada para governanca comunitaria.

A analise identificou **zero vulnerabilidades criticas ou de alta severidade**. Foram encontradas duas questoes de media severidade — ambas mitigadas antes da entrega — e tres de baixa severidade, das quais duas foram aceitas como risco controlado e uma resulta em recomendacao para producao.

Severidade	Encontradas	Mitigadas	Aceitas	Pendentes
CRITICAL	0	—	—	0
HIGH	0	—	—	0
MEDIUM	2	2	0	0
LOW	3	1	2	0
INFO	4	—	4	0
TOTAL	9	3	6	0

Conclusao: O protocolo demonstra maturidade tecnica adequada para um MVP. As protecoes de seguranca implementadas — ReentrancyGuard, SafeERC20, AccessControl e Solidity ^0.8.20 — cobrem os vetores de ataque mais criticos. As recomendacoes para producao estao detalhadas na secao 7.

1. Escopo da Auditoria

1.1 Contratos Auditados

Contrato	Arquivo	Linhas	Padrao	Complexidade
GreenToken	GreenToken.sol	52 LOC	ERC-20	Baixa
CarbonCertNFT	CarbonCertNFT.sol	85 LOC	ERC-721	Media
GreenStaking	GreenStaking.sol	130 LOC	Custom	Alta
GreenDAO	GreenDAO.sol	145 LOC	Custom	Alta

1.2 Dependencias Externas

Os contratos dependem da biblioteca **OpenZeppelin Contracts v5.0** (ERC20, ERC721, AccessControl, ReentrancyGuard, Ownable, SafeERC20) e da interface **AggregatorV3Interface** do Chainlink para consumo do feed ETH/USD na Sepolia. Ambas as dependencias sao amplamente auditadas e consideradas de alta confiabilidade.

1.3 O que NAO foi auditado

Esta auditoria nao cobre: infraestrutura de deploy (chaves privadas, CI/CD), frontend dApp, script backend interact.js, contratos da OpenZeppelin (cobertos por auditoria propria), disponibilidade do feed Chainlink, e seguranca dos metadados IPFS dos NFTs.

2. Metodologia

A auditoria seguiu quatro camadas complementares de análise, cobrindo tanto vulnerabilidades automatizadas quanto logica de negocio:

Camada	Ferramenta	Foco Principal
Analise Estatica	Slither v0.10	Reentrancy, access control, unchecked calls, integer issues
Analise Simbolica	Mythril v0.24	Execucao simbolica, integer overflow, delegatecall, tx.origin
Testes Automatizados	Hardhat v2.19	Cobertura funcional, edge cases, roles, reentrancy real
Revisao Manual	Leitura do codigo	Logica de negocio, oracle manipulation, flash loan, DAO

Classificacao de Severidade:

Nivel	Criterio
CRITICAL	Perda direta de fundos ou comprometimento total do protocolo
HIGH	Perda parcial de fundos ou violacao de invariantes criticas
MEDIUM	Comportamento incorreto com impacto financeiro potencial
LOW	Boas praticas nao seguidas; impacto limitado em condicoes normais
INFO	Sugestoes de qualidade, legibilidade e otimizacao

3. Resultados das Ferramentas

3.1 Slither — Análise Estática

O Slither analisa o bytecode e AST dos contratos em busca de padrões conhecidos de vulnerabilidades sem executar o código.

```
$ slither . --solc-remaps "@openzeppelin=node_modules/@openzeppelin"
```

Detector	Severidade	Contrato	Status
reentrancy-eth	MEDIUM	GreenStaking	MITIGADO
reentrancy-no-eth	MEDIUM	GreenDAO	MITIGADO
timestamp	LOW	GreenStaking	ACEITO
timestamp	LOW	GreenDAO	ACEITO
missing-zero-check	LOW	GreenStaking	MITIGADO
calls-loop	INFO	GreenDAO	INFORMATIVO
constable-states	INFO	GreenToken	INFORMATIVO
dead-code	INFO	GreenStaking	INFORMATIVO
solc-version	INFO	Todos	INFORMATIVO

3.2 Mythril — Análise Simbólica

O Mythril executa simbolicamente os contratos, explorando todos os caminhos possíveis para detectar vulnerabilidades como reentrancy, integer issues e controle de acesso inadequado.

```
$ myth analyze contracts/GreenStaking.sol --solc-json mythril.json $ myth analyze contracts/GreenDAO.sol --solc-json mythril.json
```

SWC ID	Nome	Contrato	Status
SWC-107	Reentrancy	GreenStaking	MITIGADO
SWC-101	Integer Overflow/Underflow	Todos	MITIGADO
SWC-115	Authorization via tx.origin	Nenhum	N/A
SWC-104	Unchecked Call Return Value	GreenStaking	MITIGADO
SWC-116	Block Timestamp Manipulation	GreenStaking	ACEITO
SWC-111	Use of deprecated functions	CarbonCertNFT	INFO

3.3 Hardhat — Suite de Testes

```
$ npx hardhat test --reporter verbose
```

```
GreenToken (4 testes) v Deploy com supply inicial de 10M GRN v Mint respeita MAX_SUPPLY (100M) v  
Apenas MINTER_ROLE pode mintar v Burn com BURNER_ROLE funciona corretamente CarbonCertNFT (4  
testes) v Mint de certificado com metadados corretos v Apenas MINTER_ROLE pode chamar  
mintCertificate v tokenURI retorna URI correto apos mint v certificates mapping populado  
corretamente GreenStaking (5 testes) v Stake e unstake de tokens v Recompensas calculadas  
corretamente por segundo v Ataque de reentrancy bloqueado pelo nonReentrant v Oracle retorna  
preco ETH/USD do Chainlink mock v Bonus APR ativado quando ETH > $2.000 GreenDAO (4 testes) v  
Proposta criada com periodo de votacao correto v Voto ponderado pelo saldo GRN v Quorum 1%  
validado na finalizacao v Double-vote bloqueado por hasVoted mapping 17 passing (4.218ms) | 0  
failing
```

4. Vulnerabilidades Identificadas

As vulnerabilidades são apresentadas em ordem decrescente de severidade. Cada finding inclui descrição técnica, impacto potencial, evidência de código e a correção aplicada ou recomendação.

4.1 Severidade MEDIUM

M-01	Reentrancy em GreenStaking.unstake()				MEDIUM	MITIGADO
Contrato	GreenStaking.sol	Funcao	unstake(uint256)	Linha(s)	L.72-9	0
Descricao	A funcao unstake() realizava a transferencia de tokens (safeTransfer) antes de zerar o saldo do usuario e emitir o evento, criando uma janela de reentrancy. Um contrato malicioso poderia chamar unstake() novamente durante a transferencia antes que o estado fosse atualizado.					
Impacto	Em tokens ERC-777 (com hooks) ou ERC-20 com callbacks customizados, um atacante poderia drenar multiplas vezes seu saldo staked em uma unica transacao, causando perda de fundos para o protocolo.					
Recomendacao	Aplicar ReentrancyGuard com modificador nonReentrant e reorganizar o codigo seguindo o padrao Checks-Effects-Interactions: atualizar todos os estados ANTES de qualquer chamada externa.					
Codigo vulneravel:						
<pre>// VULNERAVEL – transferencia antes de atualizar estado function unstake(uint256 amount) external { stakingToken.safeTransfer(msg.sender, amount); // chamada externa PRIMEIRO stakes[msg.sender].amount -= amount; // estado depois (ERRADO) }</pre>						
Correcao aplicada:						
<pre>// CORRIGIDO – Checks-Effects-Interactions + nonReentrant function unstake(uint256 amount) external nonReentrant { require(stakes[msg.sender].amount >= amount, 'insufficient'); _accumulateRewards(msg.sender); // 1. efeitos internos stakes[msg.sender].amount -= amount; // 2. atualiza estado totalStaked -= amount; // 3. atualiza estado stakingToken.safeTransfer(msg.sender, amount); // 4. interacao }</pre>						

M-02	Reentrancy em GreenDAO.execute()				MEDIUM	MITIGADO
Contrato	GreenDAO.sol	Funcao	execute(uint256)	Linha(s)	L.122-132	
Descricao	A funcao execute() realiza uma chamada arbitraria a um contrato externo (p.target.call(p.callData)) antes de garantir que o estado da proposta nao possa ser re-executado. Uma proposta com target malicioso poderia re-entrar na DAO durante a execucao.					
Impacto	Uma proposta maliciosa aprovada poderia ser executada multiplas vezes se o contrato alvo re-entrasse na DAO durante a execucao, potencialmente drenando a tesouraria ou executando acoes nao autorizadas repetidamente.					
Recomendacao	Marcar a proposta como Executed ANTES da chamada externa e aplicar ReentrancyGuard. O padrao Checks-Effects-Interactions e essencial em funcoes com chamadas externas arbitrariedades.					
Codigo vulneravel:						
<pre>// VULNERAVEL – estado atualizado apos chamada externa function execute(uint256 proposalId) external onlyOwner { Proposal storage p = proposals[proposalId]; (bool success,) = p.target.call(p.callData); // chamada ANTES p.state = ProposalState.Executed; // estado DEPOIS }</pre>						
Correcao aplicada:						
<pre>// CORRIGIDO – estado atualizado antes + nonReentrant function execute(uint256 proposalId) external onlyOwner nonReentrant { Proposal storage p = proposals[proposalId]; require(p.state == ProposalState.Passed, 'not passed'); p.state = ProposalState.Executed; // Effects PRIMEIRO if (p.target != address(0)) { // Interactions por ultimo (bool ok,) = p.target.call(p.callData); require(ok, 'execution failed'); } emit ProposalExecuted(proposalId); }</pre>						

4.2 Severidade LOW

L-01	Ausencia de Staleness Check no Oracle Chainlink				LOW	ACEITO
Contrato	GreenStaking.sol	Funcao	getEthUsdPrice()	Linha(s)	L.95-9 7	
Descricao	A funcao getEthUsdPrice() consome latestRoundData() do Chainlink sem verificar o campo updatedAt. Se o feed Chainlink ficar offline ou atrasar, o contrato continuaria usando um preco desatualizado (stale price) para calcular o APR bonus de staking.					
Impacto	Em producao, um oracle inativo poderia causar o protocolo a pagar APR incorreto por periodos prolongados. No contexto de MVP em testnet, o impacto e limitado e controlado. Risco aceito para a fase MVP.					
Recomendacao	Para producao: verificar que block.timestamp - updatedAt seja menor que MAX_STALENESS (ex: 1 hora). Implementar fallback para APR base caso o preco esteja desatualizado.					
Codigo vulneravel:						
<pre>// ATUAL (MVP) - sem staleness check function getEthUsdPrice() public view returns (int256 price) { (, price,,,) = priceFeed.latestRoundData(); }</pre>						
Correcao aplicada:						
<pre>// RECOMENDADO para producao uint256 public constant MAX_STALENESS = 1 hours; function getEthUsdPrice() public view returns (int256 price) { uint256 updatedAt; (, price,, updatedAt,) = priceFeed.latestRoundData(); require(block.timestamp - updatedAt <= MAX_STALENESS, 'Oracle: stale price'); }</pre>						

L-02	Uso de block.timestamp para Controle de Período				LOW	ACEITO
Contrato	GreenStaking.sol, GreenDAO.sol	Funcao	stake(), vote(), finalize()	Linha(s)	Múltiplas	
Descrição	block.timestamp é manipulável por validators/mineradores dentro de uma janela de aproximadamente 15 segundos. Os contratos usam timestamp para calcular recompensas de staking e o período de votação da DAO.					
Impacto	O impacto é desprezível para este protocolo: a janela de manipulação (15s) é irrelevante em períodos de dias (staking) e 3 dias (DAO). Seria crítico apenas se timestamp fosse usado para randomness ou decisões de milissegundos. Risco aceito.					
Recomendação	Manter o uso atual de block.timestamp para períodos de dias ou mais. Se precisar de maior precisão, considerar block.number com tempo médio de bloco como referência.					

L-03	Voto Baseado em Saldo Corrente (Flash Loan Attack)				LOW	ACEITO
Contrato	GreenDAO.sol	Funcao	vote(uint256, bool)	Linha(s)	L.88-100	
Descricao	O peso do voto e calculado usando govToken.balanceOf(msg.sender) no momento do voto, nao em um snapshot historico. Um atacante com acesso a flash loans poderia tomar emprestado GRN, votar em uma proposta e devolver os tokens na mesma transacao.					
Impacto	Teoricamente, um atacante com flash loan poderia manipular votos em propostas criticas. Na pratica, o GRN teria que estar disponivel em pools de liquidez (inexistente no MVP). Risco aceito para MVP; critico em producao.					
Recomendacao	Para producao: usar ERC20Votes com snapshot de votacao (herdar de ERC20Votes no GreenToken e usar getPastVotes(voter, proposalSnapshot) no momento da criacao da proposta, nao do voto).					
Correcao aplicada:						
<pre>// RECOMENDADO para producao // GreenToken: herdar de ERC20Votes // GreenDAO: snapshot no propose() proposals[pid].snapshotBlock = block.number; // vote(): uint256 weight = govToken.getPastVotes(msg.sender, proposals[proposalId].snapshotBlock);</pre>						

4.3 Severidade INFO

I-01 Uso de Counters.sol (deprecated no OZ v5)				INFO	INFORMATIVO
Contrato	CarbonCertNFT.sol	Funcao	mintCertificate()	Linha(s)	L.8, L.22
Descricao	O contrato CarbonCertNFT usa o utilitario Counters da OpenZeppelin (Counters.Counter), que foi removido na versao 5.0. A versao atual compila porque a dependencia e local, mas pode causar problemas em futuras atualizacoes de dependencias.				
Impacto	Nenhum impacto de seguranca. Questao de manutencao e compatibilidade futura.				
Recomendacao	Substituir Counters.Counter por uma variavel privada uint256 incrementada diretamente, que e o padrao recomendado no OpenZeppelin v5.				
Codigo vulneravel:					
<pre>// ATUAL – usando Counters deprecated using Counters for Counters.Counter; Counters.Counter private _tokenIds; _tokenIds.increment(); uint256 newId = _tokenIds.current();</pre>					
Correcao aplicada:					
<pre>// RECOMENDADO – variavel privada direta uint256 private _nextTokenId; uint256 newId = ++_nextTokenId;</pre>					

I-02 Ausencia de Pausa de Emergencia (Pausable)					INFO	INFORMATIVO
Contrato	GreenStaking.sol, GreenDAO.sol	Funcao	Contrato completo	Linha(s)	N/A	
Descricao	Nenhum dos contratos implementa o padrao Pausable da OpenZeppelin. Em caso de descoberta de vulnerabilidade critica apos deploy, o protocolo nao teria mecanismo de pausa de emergencia para proteger os usuarios.					
Impacto	Sem impacto direto no MVP. Risco operacional em producao.					
Recomendacao	Para producao: adicionar heranca de Pausable e o modificador whenNotPaused nas funcoes de stake, unstake, vote e execute. O owner (ou multisig) deve poder pausar o protocolo em emergencias.					

I-03	Falta de Limite de Gas em Chamadas Externas da DAO				INFO	INFORMATIVO
Contrato	GreenDAO.sol	Funcao	execute(uint256)	Linha(s)	L.128	
Descricao	A chamada p.target.call(p.callData) na funcao execute() nao especifica limite de gas. Proposta maliciosa poderia executar operacoes caras e consumir todo o gas disponivel na transacao.					
Impacto	Risco de DoS em transacoes de execucao de propostas. Sem perda de fundos direta.					
Recomendacao	Especificar limite de gas na chamada: p.target.call{gas: MAX_EXECUTION_GAS}(p.callData). Valor sugerido: 200.000 gas para a maioria dos casos de uso.					

I-04	Eventos ausentes em funcoes administrativas				INFO	INFORMATIVO
Contrato	GreenStaking.sol	Funcao	constructor()	Linha(s)	L.55-6 1	
Descricao	O construtor do GreenStaking nao emite eventos para registrar a configuracao inicial (endereco do token, feed oracle, owner). Dificulta auditoria off-chain e rastreamento de historico via indexadores.					
Impacto	Sem impacto de seguranca. Afeta observabilidade e integracao com subgraphs.					
Recomendacao	Emitir evento no construtor com os parametros de inicializacao: emit StakingInitialized(stakingToken, priceFeed, owner, block.timestamp).					

5. Checklist de Boas Praticas

Item de Segurança	Resultado	Observação
Solidity versao fixa ^0.8.x	PASS	^0.8.20 em todos os contratos
Sem uso de tx.origin	PASS	Apenas msg.sender para autenticação
SafeERC20 para transferencias	PASS	Implementado em GreenStaking
ReentrancyGuard nas funcoes criticas	PASS	nonReentrant em stake/unstake/claim/execute
Checks-Effects-Interactions	PASS	Estado atualizado antes de transferencias
AccessControl com roles explicitas	PASS	MINTER_ROLE e BURNER_ROLE no GreenToken
Sem selfdestruct	PASS	Nenhum uso em nenhum contrato
Sem delegatecall nao verificado	PASS	Nenhum uso de delegatecall
Eventos em acoes criticas	PASS	Staked, Unstaked, VoteCast, ProposalCreated
Overflow/underflow protegido	PASS	Solidity ^0.8 reverte automaticamente
Variaveis imutaveis como immutable	PASS	stakingToken e priceFeed sao immutable
Require com mensagem de erro	PASS	Todos os require tem mensagem clara
Sem magic numbers	WARN	ETH_BONUS_THRESHOLD poderia ser parametrizavel
Oracle staleness check	WARN	Ausente no MVP — recomendado para producao
Snapshot de votos (anti-flashloan)	WARN	Balanco atual — ERC20Votes recomendado
Mecanismo de pausa	WARN	Ausente — Pausable recomendado para producao
Timelock para execucao de propostas	WARN	Owner direto no MVP — Timelock recomendado
Multisig para admin	WARN	Ownable simples no MVP — Gnosis Safe recomendado

Legenda: PASS = implementado e correto. WARN = ausente ou incompleto; aceitavel para MVP, recomendado para producao.

6. Analise Detalhada por Contrato

6.1 GreenToken (ERC-20)

O GreenToken é o contrato de menor complexidade e maior dependência de segurança, pois é a base do sistema econômico do protocolo. A herança de ERC20, ERC20Burnable, ERC20Permit e AccessControl da OpenZeppelin garante que as funções críticas de transferência, approve e permit estejam corretamente implementadas.

Aspecto	Avaliação	Detalhes
Controle de Acesso	EXCELENTE	AccessControl com DEFAULT_ADMIN_ROLE, MINTER_ROLE e BURNER_ROLE. Apenas o administrador pode alterar os papéis.
Mint seguro	EXCELENTE	Verificação de MAX_SUPPLY antes de qualquer mint. Impossível criar tokens além do limite.
Burn seguro	BOM	burnFrom sobrescrito para exigir BURNER_ROLE. O burn direto via ERC20Burnable permanece.
ERC20Permit	BOM	Suporte a assinaturas off-chain (EIP-2612) para aprovações gasless. Correto e útil para UX.
Eventos	BOM	TokensMinted e TokensBurned emitidos em todas as operações relevantes.

6.2 GreenStaking (Staking + Oracle)

O GreenStaking é o contrato de maior risco financeiro, pois custodia tokens dos usuários e mintar novos tokens como recompensa. A análise identificou e mitigou o vetor de reentrancy (M-01) e o risco de oracle desatualizado (L-01). O mecanismo de acumulação de recompensas é correto: rewards são calculados e acumulados antes de qualquer alteração de saldo.

Invariante crítica verificada: em nenhuma condição a soma de todos os `stakes[user].amount` deve superar `totalStaked`. Teste manual confirmou que `stake()`, `unstake()` e `claimRewards()` mantêm essa invariante.

6.3 GreenDAO (Governança)

A DAO é o contrato de maior complexidade lógica. Os principais riscos identificados são: reentrancy em `execute()` (M-02, mitigado), flash loan attack no sistema de votos (L-03, aceito para MVP), e a centralização no papel de owner para execução de propostas.

A restrição de execução ao owner e uma decisão consciente de design para o MVP. Em produção, recomenda-se substituir pelo padrão OpenZeppelin Governor com TimelockController, eliminando o ponto central de falha.

6.4 CarbonCertNFT (ERC-721)

O CarbonCertNFT é o contrato mais simples em termos de lógica de negócio. A escolha do ERC-721 sobre o ERC-1155 é adequada pois cada certificado de carbono é único. O uso de ERC721URIStorage permite metadados por token, fundamental para armazenar `co2Tonnes` e `projectName` por NFT.

Observação: O uso de `Counters` (deprecated no OZ v5) foi identificado como I-01. Não há impacto de segurança, mas a correção é simples e recomendada.

7. Recomendacoes para Producao

As recomendacoes abaixo sao requisitos para o deploy em mainnet. Estao ordenadas por prioridade de implementacao.

Prioridade	Acao	Motivo	Esforco
P1 — CRITICO	Auditoria externa profissional	Qualquer producao com valor real	Alto
P1 — CRITICO	Gnosis Safe como admin/owner	Elimina ponto unico de falha	Medio
P1 — CRITICO	ERC20Votes + snapshot de votos	Protecao contra flash loan na DAO	Medio
P2 — ALTO	TimelockController (48h delay)	Previne execucao imediata de propostas	Medio
P2 — ALTO	Chainlink staleness check	Oracle desatualizado = APR incorreto	Baixo
P2 — ALTO	Pausable em Staking e DAO	Emergencia: pausar sem upgrade	Baixo
P3 — MEDIO	Bug bounty program	Incentivo a descoberta responsavel	Medio
P3 — MEDIO	Substituir Counters por uint256	Compatibilidade com OZ v5	Baixo
P3 — MEDIO	Limite de gas em DAO.execute()	Previne DoS por gas exhaustion	Baixo
P4 — INFO	IPFS Pinning para metadados NFT	Permanencia dos metadados	Baixo
P4 — INFO	Evento no construtor de GreenStaking	Observabilidade e indexacao	Baixo

8. Conclusão

O protocolo GreenChain apresenta um nível de segurança adequado para um MVP em testnet. As vulnerabilidades de maior severidade (M-01, M-02) foram corretamente identificadas e mitigadas antes da entrega, demonstrando maturidade no processo de desenvolvimento.

As três camadas de segurança implementadas — ReentrancyGuard, Checks-Effects-Interactions e SafeERC20 — cobrem os vetores de ataque mais críticos em contratos de staking e governança. O uso de Solidity ^0.8.20 elimina toda uma classe de vulnerabilidades de overflow/underflow.

As questões de baixa severidade e informativas são consistentes com um protocolo em fase MVP: aceitáveis para testnet, porém devem ser resolvidas antes de qualquer deploy com valor real. A ausência de mecanismos como Pausable, Timelock e ERC20Votes é uma decisão deliberada de simplificação para o escopo acadêmico.

Veredicto: APROVADO para MVP em Testnet
Não recomendado para mainnet sem as melhorias P1 da seção 7.